DEVIATHON 2.0 · DEV_103 · TEAM SOYACHAAP

OpsLens
the complete guide

Every ticket is a promise.

What we built, every decision and why, and every file explained —
in simple words, for anyone new to the project.

Source: docs/GUIDE.md · every number from eval/results.json

Written for the team and for anyone new to the project. Read it from top to bottom once; after that, use the contents to jump. Simple words on purpose. When a word is new, it is explained the first time.

Every number in this guide comes from `eval/results.json` (we call it "the results file"). If a number here and a number on the screen ever disagree, the results file wins.

Table of contents

1. The whole project in two minutes
2. Words you need
3. Follow one complaint through OpsLens
4. The problem statement, and how we answered each line
5. The big decisions — what we chose, why, and why not the other option
6. How each part works inside
7. How we test — and why you can believe the numbers
8. What we can say — the results, and what they mean
9. The real public service desk, in depth
10. What failed, and what we learned
11. The dashboard, click by click
12. The repository, folder by folder and file by file
13. How to run everything
14. How we worked
15. The timeline
16. Questions people will ask, with short answers
17. What is still open
18. If you only remember ten things

1. The whole project in two minutes

The competition

Deviathon 2.0 finale, 29–31 Oct 2026, at GLA University, Mathura. Our problem is **DEV_103**, set by Eternal Ltd (the company behind Zomato and Blinkit): *"AI-Powered Business Operations Intelligence & Decision Engine*". Our team is SoyaChaap.

The problem, in plain words

Imagine the customer-support floor of a food-delivery company. Every day about a thousand customers write in: "my order is late", "an item is missing", "I was charged but got no food", "where is

my refund?". Each complaint has a **deadline** — a promise of how fast it will be fixed. Some kinds of complaints are always late. Some teams are overloaded at dinner time. Urgent ones slip through.

The problem statement asks us to build something that:

- **finds** the recurring problems (and why they happen),
- **predicts** what will go wrong next (busy hours, complaints that will miss their deadline),
- **recommends** what to do (who should take which complaint, what to fix),
- **explains** every recommendation (why this one?),
- **measures** how much better things get.

What we built: OpsLens

OpsLens is a **decision engine for a support floor**. In one sentence: *it tells the team lead why deadlines are missed, which customers will miss theirs next, and what to do about it — and every recommendation comes with its own proof.*

Think of an experienced floor manager who:

1. reads every new complaint and knows what it is about (also in Hinglish: "khana abhi tak nahi aaya");
2. remembers every past week and spots what keeps going wrong;
3. can feel which customers are about to be let down, and why;
4. moves the right complaint to the right person at the right moment;
5. — and, unlike any human, can **replay a whole day** two ways and show which way kept more promises.

OpsLens does those five things with software, and does the fifth one carefully enough that a sceptical judge can trust it.

The headline

On **14 days OpsLens never saw**, with **the same customers and the same agents**:

- **2.0%** of customers missed their deadline with OpsLens,
- **5.5%** missed with the industry-standard way of routing work.

The difference is −3.46 points, and the 95% interval is −4.77 to −2.15 (the whole range is below zero, so it is not luck). OpsLens was lower on **14 of 14** of those days.

In everyday words: out of every thousand customers, about 55 were let down the usual way, and about 20 with OpsLens — with the same people working the same hours.

Two worlds

The dashboard has a switch at the top between two "worlds":

- **Food delivery (simulated)** — a support floor we built in the computer, with 40 agents and eight weeks of complaints. Because we built it, we know the full truth, so we can replay decisions and prove which ones help.
- **IT service desk (real, public)** — a real company's IT help desk, from a public dataset (the UCI ServiceNow log, 24,918 incidents). It shows the ideas hold on real data, not just on ours.

What makes OpsLens different (our "moats")

1. **Proof, not promises.** Every claim is tested on unseen days, with fair comparisons and ranges.
2. **Trust.** Every number on the screen is a button: click it and see where it was computed. The AI is never allowed to write a number.
3. **Touch it.** Judges can change things — "what if Team C loses two agents?", "hide a problem in the data" — and watch the result appear.

And one unusual habit: **we wrote down the rule for each experiment before running it**, and we keep the ones we failed on screen. That is the screen called *Rules we wrote first*.

2. Words you need

Word	Meaning in plain words	Example
Ticket / complaint / incident	One customer problem. "Incident" is the IT-desk word.	"Mera paisa wapas karo please" (a refund request)
Category	What the complaint is about. We use eight.	late delivery, missing item, wrong item, refund, payment failure, app bug, account, restaurant partner
Priority	How urgent it is: P1 (most urgent), P2, P3.	A late live order is P1
SLA / deadline	The promised time to fix it.	P1: 1 hour, P2: 4 hours, P3: 24 hours
Breach / missed deadline / missed promise	Fixed after its deadline. Our main score is the **share of tickets that breached**.	2.0% = 20 in every thousand
Agent	A support person.	Our floor has 40, in four teams
Team	A group of agents with the same main skills.	A: live orders. B: payments & refunds. C: order accuracy. D: account, app bugs, restaurant partners
Skill	Which kinds of tickets an agent is trained for.	An agent without the skill is 1.6× slower and causes more re-opens
Language	Which language the agent can serve.	Only three agents speak Hindi *and* handle refunds
Policy	A rule for who takes which ticket next.	"Oldest first" is a policy; OpsLens is a policy

Word	Meaning in plain words	Example
Baseline	A normal way of working we compare against.	First come first served; urgent first; the industry standard
Industry standard	Our name for the strongest normal way: one shared queue (priority, then deadline), any agent with the right skill. We even give it our AI classifier.	It missed 5.51%
Simulator	A program that plays out a whole day minute by minute.	Saturday 17 Oct, 18:00–24:00, played twice
Seed / luck	The random parts of a day (how long each job takes, who is absent). A seed fixes the luck so it can be repeated.	We run 5 seeds and average
Same luck for everyone	Every policy gets the exact same luck, so differences come only from decisions. (Technical name: common random numbers.)	Ticket 812's second piece of work always takes the same time, whoever handles it
Train / validation / test days	56 days in total. Days 1–35: learning. Days 36–42: choosing settings. Days 43–56: the final exam, never used for any choice.	The headline is from days 43–56
Held-out	Kept aside, not used for learning or choosing.	"Held-out days" = test days
95% interval	A range that shows how sure we are. If the whole range is below zero, the improvement is real.	−4.77 to −2.15
Pre-registration	Writing the rule that decides an experiment before running it, and saving it in git with a timestamp.	"Adopt only if the whole interval is below zero"
Risk	The chance a waiting ticket will miss its deadline.	0.997 = almost certain
Rescue	How much serving a ticket *now* lowers its risk. OpsLens prioritises rescue.	Risk 0.997 if it waits, 0.29 if a Hindi refund agent took it now
AUC	How well a model ranks risky tickets above safe ones. 0.5 = coin toss, 1.0 = perfect.	0.977 on our floor
Forecast band	A range for the future, not one number: "between X and Y with 80% confidence".	The 80% band covered 79.2% of real hours
Bottleneck	Hours when more work arrives than the people on shift can handle.	Team B short by up to 5.2 agents on a Saturday evening
Evidence card	One recurring problem with a number, a comparison and example tickets.	Hindi × Team B wait 5.1× longer
Playbook / fix at the source	An action that stops a problem from happening, not just handles it faster.	Cross-train two Hindi-speaking agents on refunds
Shadow mode	OpsLens only watches and warns; people still decide. How a company would try it first.	The real-desk replay
LLM	A large language model (we use free Gemini and Groq). We use it for words, never for numbers.	Understanding "kal raat refund wale kitne late honge?"

3. Follow one complaint through OpsLens

The easiest way to understand OpsLens is to follow one real complaint from our test days.

The complaint. Ticket **T060582**: "*mera paisa wapas karo please*" ("please give my money back"). It is a **P2 refund**, written in **Hindi**. Its deadline is 240 minutes.

Step 1 — What is it about? The local classifier reads the words and decides: refund. (It handles spelling variants and Hinglish because it looks at pieces of words, not only whole words.) If it had been unsure, it would have asked the LLM — but for about 98% of tickets it is sure.

Step 2 — Who can serve it? A refund belongs to Team B. It is in Hindi, so it needs an agent who speaks Hindi *and* is trained on refunds. In our floor only three agents have that combination, and none of them works at night. This is one of the problems OpsLens found in the history: *Hindi × Team B wait 5.1× longer* (336 minutes on average, against 66 for others).

Step 3 — How risky is it right now? With 113 minutes left, the risk model says **0.997** — almost certain to miss its deadline. And it says why:

- "no agent on shift can serve Hindi refund" — 40% of the push,
- "113 min left of 240 min SLA" — 23% of the push,
- "Hindi customer" — 11% of the push.

These reasons come straight from the model's own maths (TreeSHAP), not from a story written afterwards. We tested that they are the real reasons (section 7).

Step 4 — What would help? OpsLens re-scores the ticket as if a capable agent took it now: the risk would fall to **0.29**. That drop (from 0.997 to 0.29) is the **rescue**. A ticket with a big rescue is worth serving first. A ticket with 7 minutes left and no possible rescue is not — serving it first would sink two others that could still be saved.

Step 5 — Who takes it? Every few minutes, OpsLens looks at all free agents and all waiting tickets and picks the assignments with the most total value (urgency × business impact × rescue). When no Hindi refund agent is on shift, OpsLens can choose a **rescue by another team** (an agent from another team who speaks Hindi, slower but in time) — the improvement called v5.

Step 6 — What should change so this stops happening? On the *Actions* screen, the playbook "Cross-train 2 Hindi-speaking agents on payment failure and refund" is measured on the test days: alone it means **26.8 fewer missed deadlines** over two weeks. The manager can **Accept** it, see it in *Your plan*, and press **Simulate** to replay a day with it.

Step 7 — Is this better than normal? The same day is replayed under every way of working, with the same customers and the same luck. Over all 14 test days: 2.0% missed with OpsLens, 5.5% with the industry standard.

That is OpsLens in one story: **read → find the pattern → predict the risk with reasons → rescue the right ticket → fix the cause → prove it.**

4. The problem statement, and how we answered each line

The PDF asks for five things the system should do, four suggested technical parts, and four deliverables.

DEV_103 says	Our answer	Proof	Where on screen
Identify patterns in history	Evidence cards for five kinds of problems: capacity, skill gap, volume spike, partner issue, process issue	Found 5 of 5 planted problems; on the real desk, 3 of 3 findings still held on later months	Problems
Predict bottlenecks, delays, SLA breaches	A 48-hour forecast per team with an 80% band; each waiting ticket's risk with reasons	Risk AUC 0.977; on real data 0.857 vs 0.552 for the desk's own priority field	Risks
Recommend actions (urgency, capacity, business impact)	Automatic dispatch, fixes at the source, staffing changes, *Your plan*	All 18 recommendations together: −48.8 missed deadlines over the test days	Actions, Your plan
Explain decisions	Top reasons per ticket, evidence per problem, the runner-up agent for each assignment, every number clickable	The top reason changes the risk more than a random one in 98.9% of cases	Risks, every number
Measure impact (turnaround, productivity, quality)	Simulator replays, results under the three headings, intervals	2.0% vs 5.5%; 14 of 14 days	Overview → Results, Proof
Tech: ML for forecasting + anomalies	LightGBM forecast + a band; spike detection with a robust score	The 80% band covered 79.2% of hours	Risks → Next 48 hours
Tech: NLP / LLMs	Local classifier + LLM for unsure tickets; LLM understands Ask questions; LLM writes words, not numbers	Real benchmark BANKING77: 91.0%	Proof → How we tested
Tech: optimisation	Min-cost flow assignment; an exact roster solver tried	Honest: at our size a simple rule ties it (section 10)	Proof → Rules
Tech: dashboard incl. performance tracking	Six screens, Live now, day-by-day tracking	Lower than the industry standard on 14 of 14 days	Overview
Deliverable 1: a public dataset	The real IT desk on every screen	Shadow mode: 53 of 102 real misses warned in time vs 16	IT service desk
Deliverable 2: dashboard of insights + bottlenecks	The dashboard	—	everywhere
Deliverable 3: explainable recommendations	Reasons on every recommendation and assignment	98.9% faithfulness	Risks, Actions

DEV_103 says	Our answer	Proof	Where on screen
Deliverable 4: baseline vs proposed, measurable	4 baselines + published scheduling rules, intervals, 30 random worlds	Beat the industry standard in 29 of 30 worlds we did not design	Proof

5. The big decisions

For each big decision: **what we chose**, **why**, and **why not the other option**. Each starts with a one-line summary.

Decision 1 — Focus on support operations, with food delivery as the story

In one line: go deep on one kind of operation and prove it, rather than shallow on many.

- **Chose:** a decision engine for a support floor, with food delivery as the main story and a general engine underneath.
- **Why:** the problem statement's own example is support tickets ("1,000 support tickets daily… SLAs"). Eternal runs food delivery, so Hinglish complaints, lunch and dinner peaks and sale days make the story real to the judges.
- **Why not "all business data" (sales, workflows)?** Doing everything shallowly loses to doing one thing deeply and proving it. The engine works on any "work item" — something that arrives, has a type, a deadline and a skilled handler — and we show two kinds (food complaints and IT incidents).
- **Why not switch fully to the public IT data?** The team's decision: "good at general, insanely good at food". The food world is where decisions can be proved; the real desk is where we show it holds.

Decision 2 — Generate the food data, and add one real public dataset

In one line: build the world we need to test decisions, then check on real data that it holds.

- **Chose:** generate the food-delivery floor ourselves (56 days, 71,530 tickets, 40 agents) and add the real UCI ServiceNow incident log (public, CC BY 4.0 licence).
- **Why generate?** No company publishes support tickets with timestamps, agents, skills and deadlines. Without those you cannot test *decisions*. Generating also lets us **plant problems on purpose** and check whether OpsLens finds them — like a teacher hiding mistakes in an exam paper.
- **What we planted (five, one per kind of cause):** 1. *Skill gap:* only three agents can do Hindi refunds, none at night. 2. *Capacity:* Team C is thin on weekend dinners while missing/wrong-item complaints rise. 3. *Volume spike:* a payment-failure spike on one history day (a "bad app release"), and a surprise app-bug surge on a test day that no forecast knew about. 4. *Partner issue:* one restaurant cluster sends three times the normal wrong items. 5. *Process issue:* the phone menu (IVR) mislabels a quarter of call tickets, so they bounce between teams. Plus "decoys" that must **not** be flagged: the two sale days (known events) and one busy-but-normal day.

- **Why the UCI log?** It is a real company's IT desk with a real "made SLA" flag, real teams and hand-offs, and a clear licence. It answers "you only tested on data you made".
- **Why not the popular Kaggle "Zomato delivery" data?** We checked: it has no licence, and it is itself generated (delivery points shifted by exact steps, pickup times always 5, 10 or 15 minutes). Using it would hurt our credibility.
- **Ticket text:** Bitext (a public customer-support text collection) for common categories, plus our own templates (dishes, restaurants, amounts, Hinglish) for food-specific ones. We say openly that the text is partly synthetic, and we also test the classifier on a real text benchmark (BANKING77).

Decision 3 — Build our own simulator to test decisions

In one line: a decision can only be judged by replaying the day with and without it.

- **Chose:** our own small simulator (`opslens/simulate.py`).
- **Why a simulator at all?** You can score a *prediction* on history ("did it come true?"). But a *decision* changes what happens next. The only fair way to compare "who takes which ticket" is to replay the same day under each policy.
- **Why our own and not a library?** Every line must be explainable to Eternal's engineers. A few hundred lines with a queue of events is explainable; a big framework is a black box.
- **How we trust it:** tiny hand-calculated examples must come out exactly right, and a big random run must match the classic call-centre formula (Erlang C) for average wait within 5%.
- **A fix we found ourselves:** at first, a ticket sent to the wrong team got a fresh "luck draw", so a policy could, in theory, re-roll a long job by misrouting it. We fixed it (the k-th piece of real work always uses luck draw number k) and re-ran everything. Finding and fixing such things is why the numbers are believable.

Decision 4 — Strict fairness rules for every comparison

In one line: nobody can say "you tuned it to win".

- Same customers, same luck, same agents, same total hours for every policy.
- All choices on validation days; test days run once.
- When anything changed later, a **brand-new "sealed" world** was generated after freezing and run once.

Decision 5 — Compare against strong opponents, not easy ones

In one line: beating a weak rule proves nothing.

Name	How it works	Missed (test days)
A — First come, first served	each team serves its oldest ticket first	13.1%
B — Urgent first	highest priority first, then oldest	6.8%
C — SLA due	the ticket closest to its deadline first	5.38%

Name	How it works	Missed (test days)
D — Industry standard	one queue (priority, then deadline), any agent with the right skill, our AI classifier	**5.51%**
Least slack first (published rule)	the ticket with the least time to spare	3.94%
Generalised c-mu (published rule)	value per minute of work	5.27%
OpsLens	rescue-based value + optimiser	**2.05%**

- **Why D is the lead comparison:** it is the strongest *normal* way of working, and we gave it our own AI triage — so it is a tough opponent.
- **Why published rules too:** a professor will ask "did you beat the best known rules, not just helpdesk defaults?" On our floor, yes. Where one beats us — sustained overload — see section 10.

Decision 6 — Value the rescue, not just the risk

In one line: put effort where it changes the outcome.

- **Chose:** every few minutes, OpsLens asks the risk model two questions per waiting ticket — "risk if it waits?" and "risk if served now?". The difference is the **rescue**. A ticket's value = urgency × business impact (gold customers count 1.5×, bigger orders a little more) × (rescue + time already waited).
- **Why not just "riskiest first"?** The riskiest tickets are often already lost (7 minutes left). Serving them first sinks others that could still be saved.
- **Proof that this matters:** a simpler model (logistic regression) ranks tickets almost as well (AUC 0.957 vs 0.977), but used for rescue inside the dispatcher it misses 3.87% vs 2.05%. With no risk model at all, OpsLens loses 2.15 points. The prediction is the heart of the system.
- **Safety rules:**
- *No customer is abandoned:* a ticket far past its deadline gets served (the "starvation guard").
- *Crisis mode:* when the queue is very long, the guard pauses, to save what can still be saved.
- **How the policy grew (every change tested on validation days first):**
- v1 and v2 failed a stress test with 25% more volume (late tickets crowded out savable ones).
- v3 fixed that: a ticket past its deadline stops outranking savable ones.
- v4 (pricing cross-team work differently) was **not adopted** — its gain was not clearly above zero.
- **v5 (shipped):** if no trained agent who speaks the customer's language will be on shift before the deadline, a rescue by another team pays no penalty. Confirmed on 20 fresh seeds; test days 2.21% → 2.05%.
- v6 (an "overload mode") is being tested now (section 17).

Decision 7 — Use an exact optimiser, and be honest about what it adds

In one line: the matching is textbook; the win comes from the values we feed it.

- **Chose:** at each decision, solve "which free agent takes which ticket" as a **min-cost flow** problem (Google OR-Tools), maximising the total value.
- **Why:** it is the exact method for this kind of matching, fast (5,000 waiting tickets × 500 agents in about a fifteenth of a second), and handles agents with several free slots.
- **The honest part:** decisions happen the moment anything changes, so usually only one agent is free. Then the best global match equals "each free agent takes their best ticket". Our own test shows a greedy rule ties min-cost flow (−0.016 points), even with three chats per agent (+0.01). We say this first, rather than wait for a judge to find it.

Decision 8 — LightGBM for risk and forecast, not deep learning

In one line: the best tool for tables, explainable, and fast on a laptop.

- **Chose:** LightGBM (many small decision trees added together) for the risk model and the forecast.
- **Why:** it is the strongest common method for table-shaped data, runs on a laptop with no GPU, retrains in minutes, and gives exact explanations (TreeSHAP) for free.
- **Why not deep learning?** Slower, needs more data, harder to explain; the rules said free tools and no GPU. We still tested modern "foundation models" as challengers; they did not beat ours by the margin we set in advance, so we kept LightGBM.
- **Calibration:** the risk model's numbers are adjusted on validation days so that "70%" really means about 70% (its Brier score, a measure of honest probabilities, is 0.055 — lower is better).

Decision 9 — Read complaints with a cascade: local first, LLM only when unsure

In one line: cheap, fast and private for most tickets; the LLM where it actually helps.

- **Chose:** a local text model for every ticket; the LLM only for the ones it is unsure about (about 1.5%).
- **Why:** on our text the local model is 98.0% right and the cascade 98.1%. It is free, fast and nothing leaves the laptop for most tickets.
- **Why not send everything to the LLM?** On the real BANKING77 benchmark the LLM alone scored 79.0%, below our local model (90.3%), and it is slower and uses quota. The cascade scored 91.0%; with only ten examples per category it lifted accuracy from 68.5% to 78.7%.
- **Privacy:** phone numbers, emails, order IDs, card-like numbers and UPI IDs are masked before any text leaves the laptop.

Decision 10 — The AI is never allowed to write a number

In one line: one invented number would destroy all trust.

- When an LLM writes text (the brief, cluster names), it may only refer to numbers by a key, like `{{opslens_breach}}`. Our code checks the reply: any digit, number word or unknown key → rejected → a fixed template is used instead. Our code fills in the real values from the results file.

- **Ask OpsLens** follows the same idea: the LLM only *chooses a tool* (what-if, forecast, at-risk, explain a ticket, fix first, top problems, headline, plant a problem) and copies the user's own numbers; the engine computes the answer. If the LLM adds a number the user never typed, the route is rejected. If the LLM is offline, keyword rules answer (also Hinglish words like "kal", "kitne", "kaun").

Decision 11 — Forecast with a range, not a single line

In one line: staffing needs "how bad could it be", not just "on average".

- **Chose:** forecast tickets per category for every hour of the next 48 hours, with 80% and 90% bands made from past errors (the method is called conformal prediction).
- **Why:** a single number hides the risk. The band is honest — the 80% band covered 79.2% of real hours. A bottleneck is flagged when the top of the band is more than the team can do.
- **Accuracy:** total error (WAPE) 0.305, against 0.415 for "same hour last week".

Decision 12 — Staffing changes with the same people and the same hours

In one line: recommend what a team lead can actually do this week.

- **Chose:** compute how many agents each team needs per hour (the call-centre Erlang C formula), then propose moves that keep **the same people and the same total hours**: start a shift earlier or later, swap a day off.
- **Why:** "hire more people" is not a decision a team lead can make today; moving the hours they have is.
- **Honest result:** most single moves have no clear effect alone; an exact roster solver (CP-SAT) also did not beat them on missed deadlines. We show that.

Decision 13 — Fix problems at the source

In one line: the best complaint is the one that never needs to be written.

- **Chose:** each problem card maps to an action for the team that can remove it — partner operations (a restaurant's packing), product (the phone menu), training (cross-training), customer communication (a status banner, self-serve refunds). Each action is replayed on the test days to measure its effect.
- **Best one:** cross-training two Hindi-speaking agents on payment failures and refunds: **26.8 fewer missed deadlines** over the test days, alone.
- **Honesty:** any effect that is a business guess (for example, how many customers a status banner stops from writing in) is labelled as an assumption and tested over a range.

Decision 14 — Write the rules first, and show the failures

In one line: showing our misses is what makes our hits believable.

- **Chose:** before every serious experiment, write the success rule in `docs/PLAN_V4.md` (or the module's explain file) and commit it; git keeps the timestamp. Run once, report whatever happens.

- **Why:** it is the strongest answer to "you only show what worked".
- **On screen:** *Rules we wrote first* lists every rule with its date, what we hoped, what happened, and a stamp (kept / missed / only reported). A planned improvement groups it into "core claims" and "stretch bets" (section 17).

Decision 15 — Test far beyond our own world

In one line: a result that only holds in one world we built is not worth much.

- **Sealed world:** generated *after* the model was frozen, run once: 2.2% vs 5.8% for the industry standard.
- **30 random worlds:** volume, skills, deadlines, absences, team sizes and chats per agent all drawn at random from wide ranges fixed in advance. OpsLens beat the industry standard in **29 of 30**.
- **Agents with two or three chats at once:** OpsLens still wins in all four settings tested.
- **The physical floor:** we worked out, ticket by ticket, which deadlines could not be met even with unlimited agents (1.31% of tickets). OpsLens captures about 83% of the improvement that is physically possible. (Not shown on screen by the team's choice; mention it only if asked.)

Decision 16 — Say exactly what the real desk proves, and stop there

In one line: judges trust a team that draws the line in the right place. (Full detail in section 9.)

Decision 17 — Plain technology that works offline

Chose	Why	Not chosen, and why not
Python 3.12, pandas, NumPy, scikit-learn, LightGBM, OR-Tools	free, fast, standard, explainable	—
FastAPI server	small, typed, easy to explain	Django (too big for this)
Plain HTML + JavaScript + CSS, charts bundled	works with Wi-Fi off; every line readable	React (needs a build step, harder to explain); Streamlit (looks generic, less control)
Free LLM tiers (Gemini Flash-Lite, Groq Qwen), with a cache	free; cached answers keep the demo working offline	paid APIs
One results file (JSON)	one source of truth, easy to check	a database (more moving parts)

Decision 18 — A dashboard built around a manager's questions

In one line: six screens, six questions.

- What is happening? Why does it go wrong? What will go wrong? What should we do? Does it work? Will it work on my data?
- **Two worlds** with one switch, same screens.
- **Accept / Override → Your plan:** the manager decides the big things (fixes, staffing); OpsLens decides the thousand small ones (who takes which ticket) and explains each. *Accept* shows the

measured effect; *Simulate* replays a day with the plan; *Override* records why (a manager's "no" is information too).

- **Live now:** today's weekday, simulated on the laptop at the real clock — the demo is alive.
- **The look:** a neon theme with a light switch, the OpsLens mark, small characters, and an "Ask OpsLens AI" assistant. Rule for the demo: the key numbers must stay visible on a projector.

6. How each part works inside

This section opens each engine part a little more. You do not need it to use OpsLens, but it helps when a judge asks "how exactly?".

6.1 Generating the food floor (`data/generate_ops.py`)

- **Volume:** a little over a thousand complaints on a weekday, more at weekends, more on two sale days; most arrive around lunch and dinner.
- **Each complaint gets:** a category, the category the customer *picked* in the app (sometimes wrong), a channel (chat, call, email), a priority, a language, a customer tier (normal or gold), an order value, a restaurant cluster, and text.
- **Agents:** 40 people in 4 teams, each with skills, languages, a 9-hour shift and days off (`config/roster.yaml`). Start times were chosen by a small search to spread the load fairly, so the "normal" floor is reasonable, not set up to fail.
- **History:** days 1–42 are played out with Baseline A (first come, first served), so the history looks like a normal floor.
- **The truth file:** the planted problems are written to `data/planted.json`. **Only the evaluation code reads it**; the engine never does, so it cannot cheat.

6.2 Reading complaints (`opslens/classify.py`)

- The local model turns text into features made of **word pieces** (so "khana" and "khaana" look alike) and whole words, then a logistic regression picks the category with a confidence.
- It learns only from **past, resolved** tickets, using the category found after resolution.
- Tickets below a confidence threshold (chosen on validation days) go to the LLM in batches; replies are checked and cached.
- Routing by the *predicted* category instead of the customer's menu choice avoids many wrong-team transfers.

6.3 Finding patterns (`opslens/patterns.py`, `opslens/kpis.py`, `opslens/clusters.py`)

- **Slicing:** waits, misses, transfers and re-opens by team, category, language, hour, weekday, channel and restaurant cluster.
- **Where the time goes:** waiting vs being worked on vs transfers vs re-opens.

- **Spikes:** each day's count is compared with normal days using a robust score that ignores outliers; a day far above normal (score above 3.5) is flagged — unless it is a known event like a sale day.
- **Evidence cards:** each finding gets a number, a fair comparison, the number of tickets affected, a cause type, example tickets, and a note on why a known event does not explain it.
- **The cards it found (examples):**
- phone (IVR) tickets are transferred 5.1× more (13.0 vs 2.6 per 100 tickets) — the planted menu problem;
- Hindi × Team B wait 5.1× longer (336 vs 66 minutes) — the planted skill gap;
- Team C overloaded on weekend dinners (work 1.45× the agent time) — the planted capacity problem;
- a payment-failure spike on Thu 17 Sep (254 vs 117 a day) — the planted spike;
- restaurant cluster 17: 2.5× more wrong-item tickets — the planted partner issue.
- **Clusters:** inside each category, similar complaints are grouped and the LLM names each group in words.

6.4 Predicting risk (`opslens/features.py`, `opslens/risk.py`)

- **What it looks at, at a given moment:** category, priority, language, tier, how long it has waited, time left to the deadline, queue length for its team and language, how many capable agents are free or on shift, hour, weekday, recent arrival rate.
- **No peeking into the future:** a test appends future events and checks the features do not change.
- **Training:** snapshots from days 1–35 ("at 19:10 this ticket was waiting… did it end up late?"); calibrated on days 36–42; reported on days 43–56.
- **Quality:** AUC 0.976, precision-recall area 0.955; of the 10% riskiest snapshots, 99.6% really were late.
- **Reasons:** TreeSHAP splits each prediction into the push from each feature; the top three are shown in words ("no agent on shift can serve Hindi refund — 40% of the push").
- **"What would help":** re-score with "a capable agent free now" and show the drop.

6.5 Forecasting (`opslens/forecast.py`)

- Predicts tickets per category for every hour of the next 48 hours, using hour of day, weekday, yesterday and last week at the same hour, recent averages, and a flag for known events.
- The band comes from how wrong the forecast was on the validation week.
- Work per hour (tickets × handling minutes) is compared with the people on shift; where the top of the band is above capacity, a **bottleneck** is named: "Team B short by up to 5.2 agents, Sat 24 Oct 18:00–23:00".

6.6 Deciding (`opslens/optimize.py`, `opslens/policies.py`)

- **When:** the moment a ticket arrives, an agent finishes or a ticket comes back, plus a tick every 2 minutes.

- **What:** for each free agent and each waiting ticket they can serve (language is never broken), a value is computed (Decision 6). Cross-team work is allowed only to rescue a ticket in real danger, with a penalty — except in the v5 case (no capable agent coming in time).
- **Scarcity:** an agent with a rare skill (like Hindi refunds) prefers rare work, but is never left idle.
- **Matching:** min-cost flow picks the best set of pairs; each assignment carries a one-line reason and the runner-up agent ("runner-up B03, value 0.31 lower").
- **Speed:** about 0.07 milliseconds per decision in our runs.

6.7 The simulator (`opslens/simulate.py`, `opslens/world.py`)

- Events sit in a queue ordered by time: a ticket **arrives**, an agent **finishes**, a customer **comes back** (re-open), or it is time to **decide**. The clock jumps from event to event.
- Every ticket's random parts (how long each piece of work takes, whether it re-opens) are drawn in advance from its own luck stream, so every policy gets exactly the same luck.
- Agents follow their shifts and days off; a policy that tries to give work to an agent who is off shift is stopped with an error.
- Agents can hold several chats at once if configured (each extra chat slows the others).

6.8 Measuring impact (`opslens/impact.py`)

- **Turnaround:** missed share, average and 90th-percentile resolution time, longest queue.
- **Productivity:** tickets per agent-hour, transfers, busy agent-hours.
- **Service quality:** re-open rate, longest waits, fairness of load, and **lateness** (how late the late ones were — so we can see if anyone is abandoned).
- Differences against each baseline with 95% intervals across seeds.

6.9 Staffing (`opslens/staffing.py`) and fixes at the source (`opslens/playbooks.py`)

- Staffing: Erlang C turns "tickets per hour × minutes per ticket" into "agents needed", with a utilisation cap; then budget-neutral moves are searched.
- Playbooks: each one is a precise change to the test days ("wrong items from cluster 17 back to normal", "these two agents now handle refunds") replayed in the simulator for 5 seeds.

6.10 Your plan (`opslens/plan.py`), the brief (`opslens/brief.py`), Ask (`opslens/ask.py`), Live now (`opslens/live.py`)

- **Your plan:** every recommendation has a stable id; accepting one applies it to the roster or the tickets; each is measured alone on the test days, and *Simulate* replays one chosen day with all accepted items, three times with different luck, in parallel (about 20 seconds; instant if repeated).
- **The brief:** a short morning decision note: the proof against the industry standard, the next bottleneck, the biggest recurring problem, and the one fix to do first — numbers filled in by code.
- **Ask OpsLens:** understands a question, picks one tool, runs it, and shows how it understood the question.

- **Live now:** when the server starts, it simulates today's weekday for the industry standard and OpsLens, then shows the floor at the viewer's real clock, minute by minute.

7. How we test — and why you can believe the numbers

7.1 The exam idea

We split the 56 days like a school year:

- **Days 1–35 are the classes** — the models learn here.
- **Days 36–42 are the practice tests** — every setting and every weight is chosen here.
- **Days 43–56 are the final exam** — used once, only to report. Nothing is ever chosen by looking at them.

7.2 The same luck for everyone

When we replay a day under two policies, every ticket arrives at the same moment, every piece of work takes the same time, the same agents are absent. Only the decisions differ. So if OpsLens misses fewer deadlines, it is because of its decisions.

7.3 Several seeds and an interval

One replay could be lucky. So each comparison runs with 5 different "lucks" (seeds), always the same 5 for every policy, and we compute the difference seed by seed. The 95% interval says how much that difference would wobble. If the whole interval is below zero, OpsLens is really better.

7.4 Rules written first

For each experiment, the rule that decides "adopt or not", "passed or missed" is written and saved in git **before** the run. After the run we only report. This stops the most common self-deception: changing the rule after seeing the result.

7.5 Worlds we did not design

- A **sealed world** generated after the freeze, run once.
- **30 random worlds** from wide ranges fixed in advance.
- **Different working styles:** agents with two or three chats at once.
- **Real public data** for everything that can be tested on a log.

7.6 The tests in code

217 automated tests check, among other things:

- the simulator on hand-calculated cases and against the Erlang-C formula;
- that no future information leaks into the risk model;
- that personal data is masked;
- that the AI can never put a digit on screen;
- that every number with a decimal point in the README, the demo script, the Q&A, the cheat sheet and this guide exists in the current results file.

8. What we can say — the results, and what they mean

The main result (food floor, 14 test days, 5 seeds, same customers and luck)

- **Missed deadlines:** OpsLens 2.05%, industry standard 5.51%, SLA due 5.38%, urgent first 6.8%, first come 13.1%. *Meaning: with the same people, OpsLens keeps more than twice as many of the promises that are usually broken.*
- **Difference vs the industry standard:** −3.46 points, interval −4.77 to −2.15. *Meaning: not luck.*
- **Day by day:** lower on 14 of 14 days. The worst day for both was the sale day, Mon 19 Oct (3.98% vs 10.60%). *Meaning: it is not one good day lifting the average; it wins every day, most of all under pressure.*
- **Your plan:** accepting all 18 recommendations together means −48.8 missed deadlines over the test days (interval −63.1 to −34.5).

Beyond our own world

- **Sealed world** (made after the freeze): 2.2% vs 5.8%.
- **30 random worlds:** beats the industry standard in 29 of 30.
- **Two or three chats per agent:** still wins in all four settings.

Prediction

- **Risk:** AUC 0.977 (logistic regression 0.957, a "time left" rule 0.884); honest probabilities (Brier 0.055).
- **Forecast:** WAPE 0.305 vs 0.415; the 80% band covered 79.2% of hours.
- **Alerts on the real desk:** of the overload alerts raised a week ahead, 77.6% came true, and 92.8% of real overloads were alerted.

Explanations

- Faithful in 98.9% of cases: removing the top reason changes the risk more than removing a random feature.

- For the planted Hindi problem, the reasons name language or skill 98.6% of the time (63% for other tickets of the same team, because thin night coverage is a real cause for everyone).

Language and AI

- BANKING77 (real customer queries): 91.0% with the cascade.
- Ask OpsLens understands 93.3% of 60 test questions, against 61.7% for keyword rules (all 18 Hinglish ones).

9. The real public service desk, in depth

What the data is

The UCI "Incident management process enriched event log": a real company's IT help desk, from early 2016 to early 2017. 24,918 incidents and 141,712 events (every change of state: opened, assigned, waiting for the user, resolved…). For each incident the company recorded whether it **made its SLA**. About 36.6% did not.

How we use it fairly

- The incidents are sorted by time. The **first 70%** are for learning; within those, the last part is for choosing settings; the **last 30%** are the test, used once.
- The same five questions as the food floor, answered on real data.

What OpsLens can claim on real data

- **Early warning.** One hour after an incident opens, OpsLens's risk ranks the incidents that will miss far better than the desk's own priority field: AUC 0.857 vs 0.552. (The priority field is almost always the same value, "3 - Moderate", so it barely separates anything.)
- **Shadow mode on a real day.** On the busiest test weekday, Thursday 5 May 2016, 102 incidents really missed their deadline. OpsLens, watching only, had warned about **53** of them in time; the priority field about **16**.
- **Findings that last.** Three problems found in the learning months — incidents that wait on outsiders, slow pickup, a category bouncing between teams — **still held on the later months, 3 of 3**.
- **Alerts:** week-ahead overload alerts came true 77.6% of the time.

What OpsLens can claim only with a stated assumption

- **Fast-track estimate.** *If* the desk could fast-track a fifth of each day's new incidents (picked up at the 1-hour mark, everything else as it really happened), OpsLens's choice would save 108 missed deadlines; the priority field's choice 29.

What OpsLens does NOT claim on real data

- **Better routing than the real desk.** A one-line rule — "fast-track whatever is still unassigned after one hour" — saves 115, a tie with OpsLens's 108. The reason: fast-tracking only helps incidents still waiting, and that rule picks exactly those. A second, "rescue"-style picker saved 160, but its interval touched zero, so we do not claim it.
- **A replay of the real desk with our decisions.** The log does not record how long people actually worked on each incident. We built a calibrated "digital twin" anyway, with a validation gate written first: running the desk's own way of working, it had to reproduce reality. It missed 27.9% where reality missed 22.4%, so it failed, and we show no twin result.
- **Detecting problems hidden in real data.** We planted 60 random problems into the real log: spikes were found well, team slowdowns not at all; 9 of 60 overall, and a second detector did not improve it.

10. What failed, and what we learned

Every row was a rule written before the run. They stay on screen.

Experiment	What we hoped	What happened	Lesson
Policy v4	Pricing cross-team work better would help	−0.08 points, not clearly above zero → not adopted	Small ideas need strong evidence
30 random worlds	Best of all six methods in 27 of 30	19 of 30 (but 29 of 30 vs the industry standard)	All losses are floors in sustained overload; there least slack wins the count, OpsLens protects gold and late customers
Real-desk fast-track	OpsLens's pick beats every other picker	Ties a one-line "still unassigned" rule	Risk is not rescue
Rescue picker	Beats the simple rule	160 vs 115, but the interval touches zero	Close is not proven
Detector v2 (real log)	Find more planted problems	No setting met the false-alarm limit → kept v1	Real data is noisy; slowdowns are hard to see
LLM root-cause naming	The LLM names causes better than rules	61 vs 62 of 62 → ties	Use the LLM for words, not judgement
Exact roster solver (CP-SAT)	Beats the simple staffing moves	Worse on the test days → not adopted	The planning score was the limit, not the search
Real-desk digital twin	Replay the real desk with our decisions	Failed its validation gate	A log without work times cannot be replayed honestly
Min-cost flow vs greedy	The optimiser adds a lot	Ties greedy	The prediction is the win

What under "sustained overload" means. In the 30 random worlds, some floors had far more work than people for weeks (10–50% of deadlines missed). There, the published "least slack first" rule missed fewer deadlines than OpsLens. When we rebuilt those 11 worlds and looked closer, OpsLens missed **fewer gold customers in 11 of 11** and kept late customers **less late in 11 of 11**: it follows its business-impact weights and never abandons the late, while least slack wins the raw count by letting the already-late wait much longer.

The big lessons:

1. The value of OpsLens is in **predicting rescue**, not in clever matching.
2. Under **sustained overload**, a different rule wins on raw counts; switching strategy needs a trigger that does not fire on normal days.
3. On **real data**, OpsLens is strongest at **early warning**, not routing.
4. Writing the rule first protected us from fooling ourselves many times.

11. The dashboard, click by click

The top bar

- **Food delivery / IT service desk** — switches every screen between the two worlds.
- **Ask OpsLens** — type a question in plain words or Hinglish, press Enter. Try: "what if Team C loses 2 agents on Saturday?", "kal raat refund wale kitne late honge?", "which customers will miss their deadline?", "why is T060582 at risk?". Below every answer, "How I answered" shows how the question was understood.
- **Neon / Light** — the colour theme (use Light if a projector washes out dark colours).
- **Your plan** — the recommendations you accepted or overrode, their measured effects, and *Simulate*.
- **Story mode** — a guided nine-step tour for the demo.

The six screens (food world)

Screen	What you see	What you can do
Overview → Live now	Today's weekday simulated at your real clock: two floors side by side, missed promises so far, customers in danger now with reasons, OpsLens's last decisions	*Play the evening* (fast), *New luck*, quick what-ifs (Team C loses 2 agents, sale rush)
Overview → Results on the test days	The headline (2.0% vs 5.5%), "out of 1,000 customers", "follow one ticket", the brief	Click any number for its source
Problems	Evidence cards, where the time goes, clusters	*Show evidence*, **Plant a problem** (hide one in fresh data, watch OpsLens find it)
Risks → Customers about to miss	The riskiest waiting tickets, top reasons, what would help	Click a ticket's numbers

Screen	What you see	What you can do
Risks → Next 48 hours	Demand band vs capacity per team; bottleneck hours	—
Actions	Fixes at the source, staffing changes, automatic dispatch with reasons	**Accept** / **Override** (pick a reason)
Proof → Side by side	Every KPI for every policy with intervals; a replayed Saturday evening	Play / speed up the replay
Proof → Beyond our world	The 30 worlds, the chats test, where OpsLens loses and why	—
Proof → Try a what-if	Change agents per team, volume, policy	Run it (the simulator replays the day)
Proof → Rules we wrote first	Every pre-registered rule with date and stamp	Click a commit
Proof → How we tested	Every evaluation, including BANKING77 and what failed	—
Your data	Upload any helpdesk CSV and map its columns; the UCI file is ready as an example	Upload / use the example

The real desk (switch to IT service desk)

- **Overview** — the fast-track outcome with its assumption, the desk's size, where an incident's time goes.
- **Overview → Shadow mode** — a real day replayed with OpsLens watching.
- **Problems** — real findings, each marked "still true on later months".
- **Risks** — early warning and week-ahead alerts.
- **Actions** — what OpsLens would advise this desk.
- **Proof → Against the real desk** — every real-data comparison, including the ones OpsLens does not win.

Every number is a button. Click it to see its path in the results file, its definition, and its range.

12. The repository, folder by folder and file by file

Top level

File	What it is
`README.md`	The front page: the problem statement answered line by line, what we do not claim, how to try it
`CLAUDE.md`	The project plan and the working rules (hard rules, module list, milestones)
`PROGRESS.md`	The log: what was built when, the numbers, open issues

File	What it is
`Makefile`	Short commands: `setup`, `data`, `ping`, `gen`, `test`, `eval`, `serve`, `demo` (on Windows: `mingw32-make <target>`)
`requirements.txt` / `requirements.lock.txt`	Python packages (and exact versions)
`pytest.ini`	Test settings
`.env` (not in git)	Your API keys. Never commit it. `.env.example` shows the format.

`config/` — every setting in one place

File	What it sets
`calendar.yaml`	Start date (31 Aug 2026), 56 days, the train/validation/test split, sale days
`generator.yaml`	How the food floor is generated (volumes, peaks, categories, languages, handle times, planted problems). **The engine never reads this** — only the generator and the simulator do
`roster.yaml`	All 40 agents: team, skills, languages, shift start, days off (fictional names)
`sla.yaml`	Deadlines: P1 60 min, P2 240 min, P3 1440 min
`optimizer.yaml`	OpsLens's weights (chosen on validation days, then frozen)
`playbooks.yaml`	Business assumptions for the fixes at the source (labelled, with ranges)
`providers.yaml`	Which LLMs to use (Gemini, Groq, a mock for tests), limits and models

`opslens/` — the engine (each file starts with a long plain-words explanation and an example)

File	What it does
`config.py`	Loads the YAML files and `.env`
`privacy.py`	Masks personal data before text goes to any LLM
`ingest.py`	Loads and checks a ticket export; computes waits, resolution times, breaches
`kpis.py`	KPI tables by any slice (team, category, language, hour…)
`classify.py`	The classifier cascade (local model + LLM when unsure)
`clusters.py`	Sub-issues inside a category; the LLM names them
`patterns.py`	Recurring problems + spike detection → evidence cards
`features.py`	Risk-model inputs built only from what is known at that moment
`risk.py`	The risk model: chance to be late, reasons, "what would help"
`forecast.py`	48-hour forecast with bands
`staffing.py`	Erlang-C needs and budget-neutral moves

File	What it does
`roster_opt.py`	The exact CP-SAT roster (tested, not adopted)
`optimize.py`	Ticket value and the min-cost-flow assignment; the runner-up text
`policies.py`	All policies: baselines A–D, published rules, OpsLens (and its overload-mode experiment)
`simulate.py`	The discrete-event simulator
`world.py`	Builds one simulated world (tickets + roster + seed + luck)
`impact.py`	KPIs under turnaround / productivity / service quality, intervals over seeds
`floor.py`	The physical floor proof (unavoidable misses)
`whatif.py`	Re-runs one day with changes (What-if)
`plan.py`	*Your plan*: applies accepted items and measures them
`playbooks.py`	Fixes at the source as scenarios to replay
`brief.py`	The morning decision brief (LLM words, code numbers, template fallback)
`ask.py`	Ask OpsLens: question → tool → answer, numbers from code
`live.py`	Live now: today's day simulated and served minute by minute
`realdesk.py`	The real desk: cards, early warning, forecast, routing check
`realdesk_fasttrack.py`	The fast-track benchmark on the real desk
`realdesk_watch.py`	Spike / slowdown / bounce detectors on the real log
`realdesk_shadow.py`	Shadow-mode data for real days
`realdesk_twin.py`	The digital-twin attempt (failed its gate, kept for honesty)
`results_schema.py`	The contract for `results.json` (which fields exist)
`llm/`	The LLM layer: `client.py` (one entry point), `openai_compat.py` (Gemini/Groq), `mock.py` (tests), `cache.py` (saved replies), `ratelimit.py` (free-tier limits), `base.py` (shared types)
`adapters/uci_incidents.py`	Turns the UCI event log into one row per incident, plus "as of" features

`data/`

File	What it is
`generate_ops.py`	Generates the 56-day food floor and the history
`text_bank.py`	Ticket text (Bitext + our templates, Hinglish)
`make_llm_templates.py` / `llm_templates.json`	One-time extra message variety from an LLM (saved in git)
`plant.py`	Random planted problems (blind tests, Plant-a-problem)

File	What it is
`planted.json`	The truth about the planted problems — **only `eval/` reads it**
`SOURCES.md`	Every dataset, its link and its licence
`public/`	Downloaded datasets (Bitext, UCI, BANKING77)
`generated/`	The generated world (not in git): tickets, history, predictions, models, staffing plan

`eval/` — every experiment (the source of every claim)

File(s)	What it measures
`run_eval.py`	Runs the steps and assembles `results.json` + `docs/NUMBERS.md` (`--assemble` = only rebuild)
`results.json`	**The single source of every number**
`fixture_results.json` , `make_fixture.py` , `results.schema.json`	Fake results for UI work (clearly marked FIXTURE) and the schema
`parts/`	Each experiment's own result file, assembled into `results.json`
`run_classification.py` , `run_patterns.py` , `recall.py` , `run_blind.py`	Reading complaints, finding problems, planted and blind recall
`run_risk.py` , `run_forecast.py`	Prediction quality and staffing plans
`run_baselines.py` , `run_opslens.py` , `run_baseline_c.py` , `run_best_baselines.py`	The main comparisons
`run_ablations.py`	What each part of OpsLens is worth; stress tests
`run_design_v2..v6.py` , `run_confirm_v5.py`	Policy changes, designed on validation days and confirmed
`run_floor.py`	The physical floor
`run_sealed_world.py`	Brand-new worlds, run once
`run_worlds.py` , `run_world_compare.py`	The 30 random worlds
`run_concurrency.py`	Agents with two or three chats
`run_tracking.py`	Day by day, team by team
`run_playbooks.py` , `run_plan_items.py`	Fixes at the source and *Your plan* items
`run_realdesk.py` , `realdesk_plants.py` , `run_realdesk_twin.py`	The real desk, its blind plants, the twin
`run_nlp_bench.py` , `run_llm_eval.py` , `llm_eval/`	BANKING77, LLM root causes, Ask routing (60 questions written first)
`run_w7.py` , `run_w8.py`	Strong baselines for prediction; explanation faithfulness

File(s)	What it measures
`run_roster_opt.py`	The CP-SAT roster
`challengers/`	Optional: foundation-model challengers (separate requirements)

`server/`

File	What it does
`app.py`	The web server: serves the dashboard and `results.json`; live endpoints What-if, Your plan, Live now (prepared at start-up), Ask, Upload
`demo.py`	`make demo`: starts offline, refuses fake (fixture) results
`plant_demo.py`	Live Plant-a-problem
`upload.py`	Analysis of an uploaded CSV

`web/` — the dashboard (plain HTML/JS/CSS)

File	What it does
`index.html`, `app.js`	The page and its start-up
`styles.css`, `theme.css`, `neon.css`	Looks: base, layout, the neon theme
`js/data.js`, `js/evidence.js`, `js/format.js`	Read numbers by path; click-to-evidence; number formats and definitions
`js/router.js`, `js/ui.js`, `js/charts.js`, `js/motion.js`	Screens and tabs; building blocks; charts; numbers that count up
`js/plan.js`	The *Your plan* panel
`js/ask.js`	The Ask OpsLens bar
`js/story.js`, `js/storycards.js`	Story mode; "Out of 1,000 customers" and "Follow one ticket"
`js/replay.js`, `js/floormap.js`, `js/intro.js`, `js/mission.js`	Replays, the floor picture, the opening scene, the mission-control band
`js/liveticker.js`	The small live panel in the side menu
`js/splash.js`, `js/logo.js`, `js/season.js`, `js/mascot.js`, `js/characters.js`, `js/scenes.js`, `js/chase.js`, `js/fest.js`	The opening splash, the OpsLens mark, the assistant character and small cartoon scenes
`js/themeswitch.js`	Neon / Light
`js/tabs/*.js`	One file per screen (see section 11)
`vendor/`	Chart.js and fonts, bundled so it works offline

`tests/` — 217 tests

See section 7.6.

`docs/`

File	What it is
`GUIDE.md` / `GUIDE.pdf`	This guide
`OWNER_CHEATSHEET.md`	One page to learn by heart before judging
`DEMO_SCRIPT.md`	The 7-minute demo, following Story mode
`JUDGE_QA.md`	80+ hard questions with short answers
`NUMBERS.md`	Every number, generated from `results.json`
`PRODUCTION.md`	How a real company would deploy it (shadow mode first), and the limits
`PLAN.md`, `PLAN_V3.md`, `PLAN_V4.md`, `CLAUDE_v1.md`	The plans over time and every pre-registered rule
`OPSLENS_IN_PLAIN_WORDS.md`	An earlier short explainer
`explain/*.md`	One plain-words explanation per module, each with a worked example
`collab/`	Notes between the two AI coding assistants

`scripts/` and `cache/`

- `download_data.py` (Bitext, UCI), `download_banking77.py`, `ping_llm.py` (checks your LLM keys work).
- `cache/` holds saved LLM replies, so the demo works offline and nothing is paid for twice.

13. How to run everything

```
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements.txt
copy .env.example .env        # optional free Gemini / Groq keys
mingw32-make data             # download the public datasets
mingw32-make gen              # generate the food floor (a few minutes)
mingw32-make test             # all tests
mingw32-make eval             # every experiment → results.json (long)
mingw32-make demo             # the dashboard, offline: http://localhost:8000/
```

Useful single commands:

- Rebuild `results.json` from the part files without re-running anything: `.venv/Scripts/python.exe eval/run_eval.py --assemble --keep-brief`

- Run one experiment: `.venv/Scripts/python.exe eval/run_<name>.py`
- Start the server while developing: `mingw32-make serve`

Before a demo:

- start the server five minutes early (Live now prepares today's day);
- press Esc to skip the splash;
- check Neon vs Light on the projector;
- run one What-if and one *Simulate* in *Your plan* as a warm-up.

If something goes wrong:

- *Live now says "Starting…"* — wait half a minute, or show *Results on the test days*.
- *Ask is slow or offline* — keyword rules still answer the common questions.
- *The server stops* — start `mingw32-make demo` again.

14. How we worked

- **The team** made every decision: what to build, what to claim, what to show.
- **Two AI coding assistants** did much of the building, under strict rules: "Claude 1" (the engine, experiments, reviews and merges) and "Claude 2" (reviews, the dashboard, several experiments). They communicate through **GitHub issues and pull requests** on the private repository `ChunChun451/opslens`.
- **The rules they work under** (in `CLAUDE.md`):
- free tools only;
- every number from `results.json`;
- the LLM never writes a digit;
- no tuning on test days;
- fair comparisons;
- the engine never reads the planted truth;
- privacy masking;
- every module explained in plain words.
- **Every merge is reviewed:** every value in `results.json` is compared before and after (no existing number may move unless on purpose), all tests must pass, and the rule must have been written before the run.
- **Why say this openly:** judges will see the AI in the commit history. The honest framing is a strength — the AI worked under rules that make its work checkable, and the team can explain every part.

15. The timeline

Date	What happened
15 Oct	Plan v2; repository, LLM layer, privacy, datasets; generator, simulator, baselines
16–17 Oct	Classifier, patterns, risk, forecast, optimiser, OpsLens policy; fixes at the source
17–18 Oct	The second AI assistant joins: review, baselines C and D, Ask OpsLens, the opening scene, sealed world
18 Oct	Simulator fairness fix; floor proof; v4 not adopted; **v5 shipped (2.05%)**; new sealed world
19 Oct	Plan v3: two worlds, six screens, *Your plan* with real Accept/Override, the real desk
19 Oct	Plan v4 (aim: top marks on every judging criterion); chats test passed; 30 worlds (19 of 30)
22 Oct	BANKING77, strong baselines, faithfulness, CP-SAT (not adopted), the ledger screen
23 Oct	Real-desk second looks (both missed); diagnosis of overload losses (gold and late protected)
24 Oct	Live now and the neon look; every proof on screen; shadow mode; the twin (failed its gate); v6 rounds; a fresh-judge review: day-by-day tracking, Ask forecasts, decision brief, new README, demo script, cheat sheet, this guide
26 Oct (evening)	Freeze
27 Oct	Rehearsal, backup video
29–31 Oct	Finale

16. Questions people will ask, with short answers

Is the data fake? The food floor is simulated, because no company publishes support data with agents and deadlines. That is why every screen also runs on a real company's public IT desk, and the classifier is also tested on a real text benchmark.

Did you build the simulator so OpsLens wins? Every policy gets the same customers and the same luck; the test days were never used for choices; a world made after the freeze gave 2.2% vs 5.8%; and 30 random worlds gave 29 of 30 wins against the industry standard.

Where does OpsLens lose? Under sustained overload (a published rule misses fewer deadlines, while OpsLens protects gold and late customers), and at routing on the real desk (a simple rule ties). We show both.

Is the optimiser needed? At our size a greedy rule ties it. The win is the rescue value from the risk model; without risk scores OpsLens loses 2.15 points.

Where is the AI? The risk model is the win (a simpler model inside the dispatcher misses 3.87% vs 2.05%); LightGBM forecasts with honest bands; a text cascade with an LLM for hard cases; an LLM that understands questions in English and Hinglish.

Does the AI make up numbers? It cannot: it only picks tools or writes keys; any digit it writes is rejected.

How do you know the explanations are real? We removed each ticket's top reason: the risk changed more than when removing a random feature in 98.9% of cases.

What does Accept do? It puts the recommendation into *Your plan* with its measured effect; *Simulate* replays a day with all accepted items; *Override* records the reason.

How would a company use it? Behind an AI first line, on the escalated tickets. Week one in shadow mode (it only watches and warns), then assist (people accept or override), then automatic for low-risk work.

Can it handle more than one chat per agent? Yes; with two or three chats at once it still wins in all four settings tested.

Did you tune on the test days? No. Choices on validation days; test days run once; every rule saved in git before the run — the dates are on the *Rules we wrote first* screen.

Why not deep learning? Tables, a laptop, explainability and speed all favour LightGBM; foundation-model challengers did not beat it by the margin we fixed in advance.

17. What is still open

- **v6 overload mode:** round 1 chose "value per minute of work in overload" (no worse than v5 on fresh seeds); round 2 (switching to least slack) failed its first gate; the final test of round 1's choice on 30 new worlds is finishing. Expected: it will not reach the 27-of-30 bar, and goes on the ledger as missed, with its numbers.
- **Screen fixes (issue #19):** a one-line headline on the first screen; the day-by-day chart; the decision brief on Overview; Ask suggestions; deep links; fewer numbers per screen; and the ledger grouped into **core claims (all kept)** and **stretch bets**.
- **How to talk about AI help:** agree as a team, then say it first.
- **Rehearsal:** the cheat sheet, two full runs of the demo script, and the backup video.

18. If you only remember ten things

1. OpsLens finds **why** deadlines are missed, predicts **who** will miss next, decides **what to do**, and **proves** it on days it never saw.
2. **2.0% vs 5.5%** missed against the industry standard; lower on **14 of 14** days.
3. Same customers, same luck, same agents — choices on validation days only, test days run once.
4. It wins because it values the **rescue** (how much serving now helps), not just the risk.
5. The optimiser itself ties a greedy rule — the **prediction** is the win. Say it first.
6. Beyond our world: **29 of 30** random worlds; a sealed world made after the freeze; agents with two or three chats.
7. Real public data: early warning **0.857 vs 0.552**; shadow mode **53 vs 16** real misses warned in time.
8. Where it loses: **sustained overload** (least slack wins the count; OpsLens protects gold and late customers) and real-desk **routing** (a simple rule ties). We show both.
9. The AI never writes a number; every number clicks through to its source.
10. We wrote every rule **before** the run, and the misses are on screen. That is why the hits are believable.